

Introduction to Automation Testing:

- Testing the software application is an important and essential part of SDLC.
- In the current world scenario, the demand for delivering the software application on time is increasing day by day, by maintaining the highest standards of quality.
- To fulfill this challenges, test automation plays an important role in SDLC.
- Test Automation reduces the need of human interaction and helps the tester to carry out testing in more effective manner by saving time from repetitive and boring task.

Q] What is Automation Testing:

Automation means execution of a program without interaction of human beings. by using an automation tool or any other third party application / tool is called as "Automation Tool".

⇒ The main objective of test automation is:

- 1) Testing Better.
- 2) Faster
- 3) Reliable.
- 4) Time Saving.
- 5) Accuracy Improvement.
- 6) Reducing the human generated error.

Q. Why testing is necessary/why should we do testing?
Testing is done in a system because we all make mistakes, some of the mistakes are not important, but some of them are expensive and danger.

Therefore, we need to identify the bug or defect in the software application after making the application because humans make mistakes all the time.

Q. How to test?

Testing an application can be done in the following ways:

- 1) Manually.
- 2) Automation. (Selenium, Load Runner, QTP).
- 3) Combination of both manual and automation.

Q. Why is Automation Testing required?

Manual testing is not accurate at all times due to human generated error.

(i) Manual testing is time consuming.

(ii) Automation testing reduces the overall testing cost and human effort.

(iii) Test Automation does not require human interaction, we can run the automated test without attending.

(iv) Automation testing improve accuracy and reduces the human generated error.

(v) Automation increases the speed of test execution. It can perform testing much faster than the human.

(vii) It supports the execution of repeated and parallel test cases.

(viii) Automation testing is more reliable than manual testing.

Q: What is Selenium.

It is an automation tool which is used to automate web based application.

It is a collection of libraries which consists of In-built classes and methods.

** Selenium Server link: www.selenium.dev.

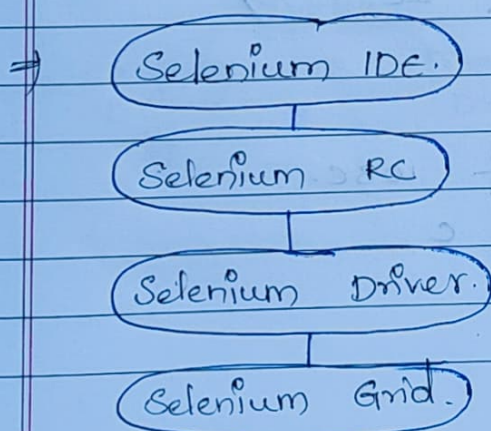
Q: Advantages of Selenium:

- It is a free and open source.
- It supports 14+ programming languages.
- It supports all major browsers.
- It supports all the major operating system.
- It allows 3rd party integration tool.

Ex! TestNG, AutoIT, Cucumber.

Q: Disadvantages of Selenium:

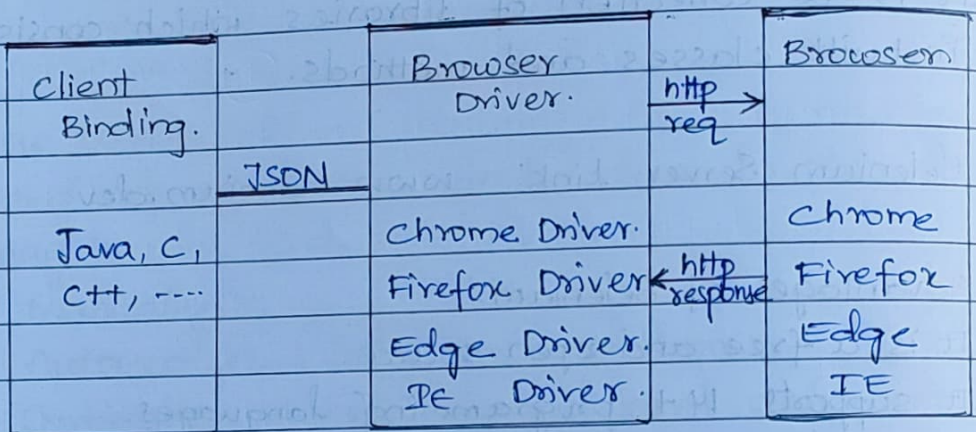
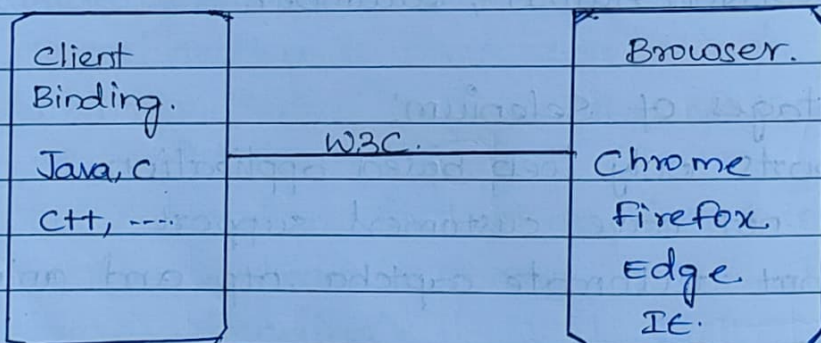
- It supports only web based application.
- It does not have customer support.
- We cannot automate captcha, otp and animation.



Selenium Architecture:

It contains four components:

- 1) client Binding.
- 2) Protocol
- 3) Browser Driver.
- 4) Browser.

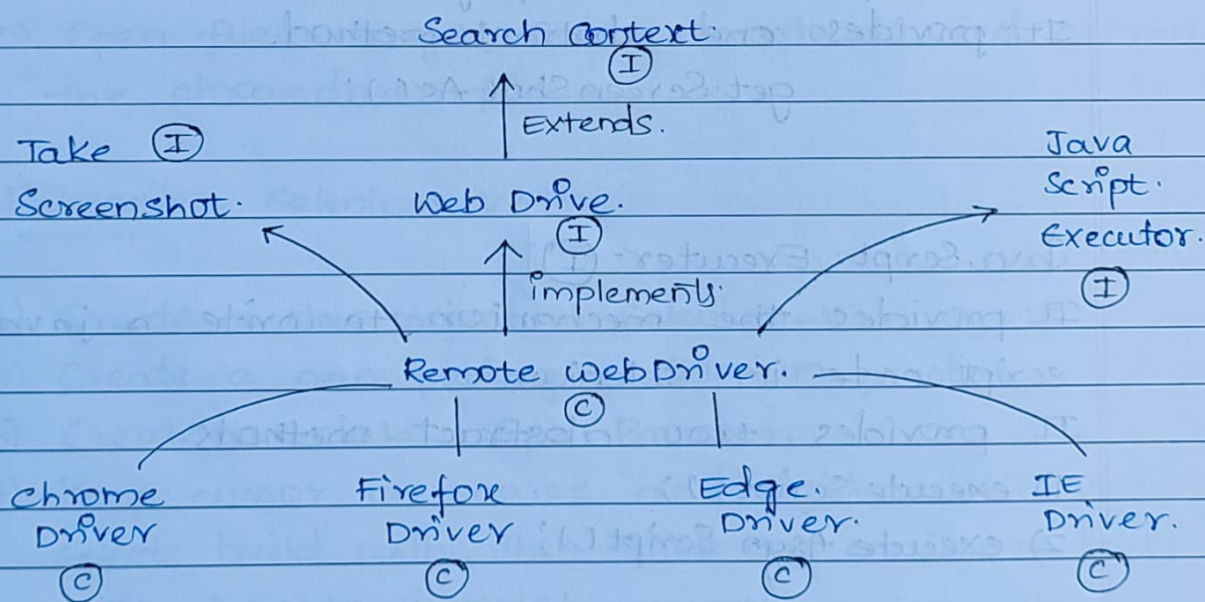
Selenium Version 3:Selenium Version 4:NOTE:

No Browser Driver because of W3C.

W3C: world-wide-web c

- Script written in any programming language should be bind with selenium this step is called client binding or language binding.
- Since browser understand only http request the script is transferred through JSON/W3C. via protocol which will convert our script into browser understandable language. (http request).
- Each browser vendors provide their own drivers to communicate with the vendors.
- When a browser driver receives request that will be send to the browser in the form of http request
- Browser perform specified action and it will be sent back by the response in the form of http response.

Selenium hierarchy : Web Driver.



Search Context:

The purpose of creating the search context Interface is, it provides the mechanism of search the web element in web page.

It is having two abstract methods:

- 1) findElement (byte);
- 2) findElements (byte);

Web Driver: (I)

It provides a mechanism to perform browser related action.

It provides an abstract methods and inherited methods.

Take Screenshot: (I)

It provides the mechanism to take a screenshot of entire web page.

It provides one abstract method.

getScreenshotAs ();

JavaScript Executor: (I)

It provides the mechanism to write a java script code in selenium.

It provides two abstract methods

- 1) executeScript ();
- 2) executeAsyncScript ();

25/03/24

How to download Selenium. (version 3).

- 1) Navigate to selenium.dev website.
- 2) click on download option. then scroll page till previous release and click on link.
- 3) Selenium 3.141.59 click.
- 4) click on selenium.server.standalone.

How to download browser driver:

- 1) Navigate to selenium.dev website.
- 2) click on download option. then scroll page till platforms supported by selenium. click on browsers.
- 3) Click on chrome. go to latest chromedriver and click on the chrome for testing availability.
- 4) Scroll and you will get stable click on chromedriver winx64, Copy the link of it and paste it in new tab, it'll get downloaded.
- 5) Open file manager, Go to downloads and extract the chromedriver file.

Eclipse: Selenium V3.

- 1) Create a New project :- Selenium-V3.
- 2) Create a new package - Selenium-practice.
- 3) Create a class - Open Browser.
- 4) Keep cursor on source and right click and select build path, then click on configure build path → Go to libraries.
- 5) Click on class path, click on ^{add} external jar. Select the file. selenium 3.141 jar file. → click on open → click on Apply → click on Apply and close.

→ Reference library is created.

6) Keep cursor on project and right click and then click on new and create a new folder name it as "chromedriver".

7) Drag and drop the chromedriver folder which is extracted, in the chromedriver folder and click on "OK".

8) To check the openBrowser we need to write the program.

→ package Selenium.practice;

default object by creation.
import org.openqa.selenium.chrome.ChromeDriver;

```
public class OpenBrowser {
    public static void main (String [] args)
    {
        System.setProperty ("driverpath", "path");
        //driver path - webdriver.chrome.driver.
        //path -
```

```
ChromeDriver driver = new ChromeDriver();
```

```
}
}
```

↓
After creating this we will get compile time error.

So, we import the statement.

NOTE! Before running the program, delete the "module-info.java" file present in src folder.

③ How to find path:
click on chromedriver folder.



Click on. chromedriver-win64.



Expand again chromedriver-win64.



Right click on. chromedriver.exe.



Click on properties.



Copy the Location path.

Eclipse: Selenium v4:

- 1) Create new project
 - 2) Create new package
 - 3) Create new class
 - 4) Delete. module-info.java file
 - 5) Keep cursor on source and right click and click on build path → configure build path → go to libraries.
 - 6) Click on classpath, click on add external jar.
Select the selenium click on open → click on apply → click on apply and close.
- Reference library is created.

⇒ Program:

package Selenium-practice;

```
import org.openqa.selenium.chrome.ChromeDriver;  
public class OpenBrowser  
{
```



```

public static void main (String[] Args)
{
    ChromeDriver driver = new ChromeDriver();
}
}

```

~~Webdriver~~

WEBDRIVER: ①

1. Non-static abstract method (direct)
2. Non-static abstract method (Inherited from search context ①)

① ⇒ get()

get Current Url()

getTitle()

getPageSource()

getWindowHandle()

getWindowHandles()

manage()

navigate()

switchTo()

close()

~~quit()~~ quit();

② ⇒ findElement()

findElements()

} → Search Context: ①

1) get();

get() help you to navigate to the particular URL or particular webpage.

return type of get() is "void".

get() will accept string type of argument.

program: {

WebDriver driver = new ChromeDriver();

Thread.sleep(4000);

Delay.

between
steps

driver.get("https://selenium.dev/");

}

2) getCurrentUrl();

getCurrentUrl(); helps you to get the url of the current webpage.

return type of the getCurrentUrl(); is String.

It will accept No Arguments

program:

{

WebDriver driver = new ChromeDriver();

Thread.sleep(2000);

driver.get("https://selenium.dev/");

String url = driver.getCurrentUrl();

System.out.println(url);

}

Output:

https://selenium.dev/

③ getTitle();

getTitle() method will help us to get the title of the webpage.

return type of getTitle() is String.

It accepts no Arguments.

Program:

```
{  
    WebDriver driver = new ChromeDriver();  
    driver.get("https://selenium.dev/");  
    String title = driver.getTitle();  
    System.out.println(title);  
}
```

④ getPageSource();

This method will help us to get the page source of the webpage.

return type of getPageSource() is String.

It accepts No Arguments.

Program:

```
{  
    WebDriver driver = new ChromeDriver();  
    driver.get("https://selenium.dev/");  
    String psource = driver.getPageSource();  
    System.out.println(psource);  
}
```

Output:

(5) `getWindowHandle()`;

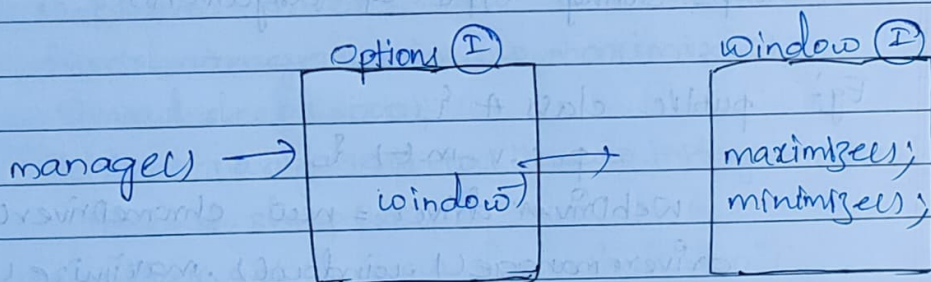
- This method is used to get the address of the current window/parent window.
- The address will be in the form of alphanumerical value.
- The return type of this method is String.
- It is an No Argument Method.

Eg:

```
public class A {
    p.s.v.m() {
        WebDriver driver = new ChromDriver();
        driver.get("URL");
        Sopln(driver.getWindowHandle());
    }
}
```

Syntax:

`driver.manage().window().maximize();`



`driver.manage().window().minimize();`

Eg:

```
public class A {
    p.s.v.m() {
        WebDriver driver = new ChromDriver();
        driver.manage().window().maximize();
        driver.get("URL");
    }
}
```


(7) navigate()

This method is used for navigation purpose

Eg: navigate to url backward action, forward action and refresh action.

- The return type of navigate method is navigation
- navigation is an interface
- This navigation Interface contains 4 methods

(1) to()

(2) forward()

(3) backward()

(4) refresh()

navigation (I)

to()

forward()

backward()

refresh()

→ to();

- It helps us to navigate to the application's particular url.
- Its return type is void.
- It accepts String type of arguments.

Eg: public class A {

 p.s.v.m() {

 WebDriver driver = new ChromeDriver();

 driver.manage().window().maximize();

 driver.navigate().to("URL");

 }

}

→ forward();

- It is used to move forward.
- The return type is void.
- It is no argument method.

Syntax: `driver.navigateForward();`

⇒ `back();`

- `back()` use to come back.
- The return type is void.
- It is no Argument method.

Syntax: `driver.navigate().back();`

⇒ `refresh();`

- It is used to refresh the web page.
- The return type is void.
- It is no Argument method.

Syntax: `driver.navigate().refresh();`

Eg:

```
public class A {  
    p.s.v.m () {  
        WebDriver driver = new ChromeDriver();  
        driver.manage().window().maximize();  
        Thread.sleep(2000);  
        driver.navigate().to("www.instagram.com");  
        Thread.sleep(2000);  
        driver.get("www.shoppystack.com");  
        Thread.sleep(2000);  
        driver.navigate().back();  
        Thread.sleep(2000);  
        driver.navigate().forward();  
        Thread.sleep(2000);  
        driver.navigate().refresh();  
    }  
}
```


- get() navigate()
- get() helps us to navigate to particular URL
 - navigate() helps us to navigate to the URL and also to do forward, back and refresh action.
 - get() will wait until all the web elements are loaded.
 - navigate() will not wait until all the web elements are loaded.

(8) close()

It is used to close the current web page.

The return type of close is void.

It is a no argument method.

Syntax: driver.close();

Eg:

```

public class A {
    p.s.v.m (-) {
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().window().minimize();
        driver.get("url");
        driver.close();
    }
}

```

→ Socket Exception:

We get this exception whenever there is a mismatch of the ~~code~~ connection.

(9) Quit();

It is used to close the current web page and also the multiple tabs.

It return type is void().

Syntax: driver.quit();

Eg: public class A {
 p.s.v. m() {
 WebDriver driver = new ChromeDriver();
 driver.manage().window().maximize();
 driver.get("URL");
 driver.quit();
 }
 }

(10) SwitchTo();

- It is used to switch to the new tab.
- Return type is target locator.
- It is a no argument method.

Syntax:

driver.switchTo().newWindow(WindowType.TAB);

Eg: public class A {
 p.s.v. m() {
 WebDriver driver = new ChromeDriver();
 driver.manage().window().maximize();
 driver.get("URL");
 Thread.sleep(2000);
 driver.switchTo().newWindow(WindowType.TAB);
 driver.get("URL");
 driver.quit();
 }
 }

(ii) findElement(By)

- It is used to find the web element in the web page.
- The return type of find element is web element.
- findElement is always give you the first matching of the web element.
- findElement method will accept 'By' type of arguments.

By:

- It is an abstract class.
- It provides 8 static methods.

- (i) Id()
- (ii) name()
- (iii) className()
- (iv) linkText()
- (v) Partial linkText()
- (vi) tagName()
- (vii) CSSSelector()
- (viii) X-path()

- All these methods accept string type of arguments.

⇒ Locators:

- Selenium is providing 8 type of locators.
- It is used to identify the location of the web element in the web page.

Types of Locators:

- | | | | | | |
|--|---|-----------------|---|---|-------------|
| <ul style="list-style-type: none"> i) id ii) name iii) class name iv) LinkText v) partialLinkText | } | Direct locator. | <ul style="list-style-type: none"> vi) Css Selector vii) X-path | } | Expression. |
|--|---|-----------------|---|---|-------------|

NOTE:

All these methods are present in By class and they are static methods which will take string type of argument.

Example:

- (1) By.id (" ");
- (2) By.name (" ");

(1) id();

It is used to identify an element by using id attribute present in the html code.

Syntax: driver.findElement(By.id(" "));

Ex:

```
<a id = "username"> </a>
```

```
driver.findElement(By.id("username"));
```


name());

° It is used to identify an element using name attribute present in the html code.

° It is given second priority.

Syntax:

driver.findElement(By.name(" "));

(3) linkText()

° It is used to identify an element by using text which is present between anchor tag.

Syntax:

driver.findElement(By.linkText(" "));

Ex:

<a> Register

driver.findElement(By.linkText("Register"));

° It is given third priority.

(4) className()

° It is used to identify an element by using class attribute which is present in html code.

° It is last recommended because it is dynamic in nature.

Syntax:

```
driver.findElement(By.className(" "));
```

Ex!

```
<a className = "ico-login"> </a>
```

```
driver.findElement(By.className("ico-login"));
```

(5) PartialLinkText()

- we go for partialLinkText only when the link is very long.

Syntax:

```
<a class = "ico-shopping cart" shopping cart </a>
```

```
driver.findElement(By.partialLinkText("Shopping"));
```

(6) Css Selector:

User Name:	
Password:	

```
<html>
```

```
<head>
```

```
<input type = "username"> UserName </input>
```

```
<input type = "password"> Password </input>
```

```
</head>
```

```
</html>
```


• In this html snippets/code to identify the username and password field. we cannot able to use id locator, name locator, link text, classname as well as partial link text.

• To identify this elements there are no such attributes attached in this input field.

• In such scenario we can use the CSS Selector.

Syntax:

"tagname [attribute Name = 'attribute value']"

"input [type = 'username']" → CSS expression for username text field.

"input [type = 'password']" → CSS expression for password text field.

WEB-ELEMENT <I>

- 1) `sendKeys()`;
- 2) `click()`;
- 3) `submit()`;
- 4) `clear()`;
- 5) `getTagName()`;
- 6) `getAttribute()`;
- 7) `getText()`;
- 8) `getCssValue()`;
- 9) `getSize()`;
- 10) `getLocation()`;
- 11) `getRect()`;
- 12) `isDisplayed()`;
- 13) `isEnabled()`;
- 14) `isSelected()`;

⇒ `sendKeys(" ")`;

◦ It is used to enter the data inside the text field.

◦ It accept String as an Arguments.

accept `charSequence`

accept `enum` as an Arguments.

◦ We have in-built enum called `keys`.

◦ Inside the `keys` enum, we have all the keyboard buttons.

⇒ `click()`;

◦ It is used to perform click action on a web element.

◦ Return type of `click()` is `void`.

→ Submit();

- It is also used to perform click action on a web element.

- Return type of Submit() is void.

- It must satisfy the following conditions.:

- i) Attribute must contain.

type="submit"

→ clear();

- It is used to clear the content which is entered into the text field.

- Return type of clear() is void.

- It is a no argument method.

Q24.

getText();

- It is used to capture the text function of the element.

- The return type of getText() is String.

- It is used for validation purpose.

- It is a no argument method.

getAttribute()

- It is used to capture the value of an attribute.

- The return type of getAttribute method is String.

- It accepts string type of an argument.

getTagName();

- It is used to capture the tagname of the web element.
- The return type of getTagName() is String.
- It is a no argument method.

05/04/12.

getCssValue()

- It is used to capture the Css value
Eg: Color, font, fontsize:
- The return type of getCssValue() is String.
- It will accept string type of arguments.

Syntax:

driver.findElement(By.id("id value")).getCssValue("Color");

06/04/12.

getSize();

- It is used to capture the size of an web element.
- The return type of getSize() is dimension.
- "Dimension" is a class
- Dimension class contains the method of
getHeight();
getWidth();

getLocation()

- It is used to capture the x and y co-ordinate in pixels
- The return type of getLocation is Point.
- Point is a class

◦ point class contains

→ getX();

→ getY();

getRect();

◦ It is used to capture the height, width, x & y locations.

◦ The return type of getRect() is Rectangle.

◦ Rectangle is a class which contains

→ getX();

→ getHeight();

→ getY();

→ getWidth();

NOTE:

We should import the Point class, Dimension class and Rectangle class by.

Org. Openqa. Selenium;

isDisplayed()

◦ To check whether the web element is displayed or not.

◦ Return type is boolean

isSelected();

◦ It is used to check whether the web element is selected or not.

◦ The return type is boolean.

isEnabled();

◦ It is used to check whether the web element is enabled or not.

◦ The return type is boolean.

X-path();

X-path is a locator which is used to identify the element by using X-path expression.

(ii) This locator works with all the attributes as well as visible text.

(iii) There are two types of X-path

1) Absolute Xpath

2) Relative Xpath.

Absolute Xpath()

• It is denoted by '/' (forward slash).

• In absolute Xpath, we start from starting of node to the end of node where we want to perform action.

• In Absolute Xpath it is mandatory to draw html tree, which is time taking process.

• Chances of error is more because of html ~~page~~ tree.

• It is totally dependent on one tag to another tag. So, In future if any other tag is added in btwn another tag. Our Absolute Xpath will not work.

Ex:

```
<html>
```

```
<head>
```

```
<body1 id="body1-value"></body1>
```

```
<body2 id="body2-value"></body2>
```

```
<input1 id="input value"></input1>
```

```
<input2 id="input value"></input2>
```

```
<span1 id="span value"></span>
```

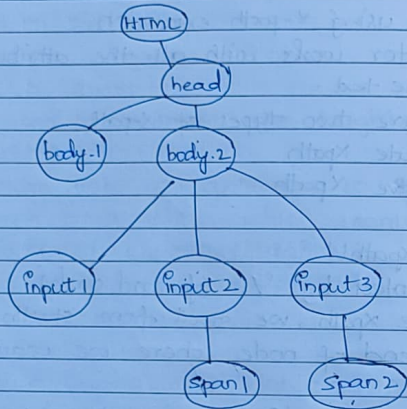
```
<input3 id="input value"></input3>
```

```
<span2 id="span value"></span2>
```

```
</head>
```

```
</html>
```


HTML - CODE



→ Example:

Traversing: from `<html>` to `<span>`

`/html/head/body/body2/input2/span1`

→ Relative Xpath.

- It is denoted by `'//'`
- In Rx directly we can reach node where we want to perform action
- In Rx we have Basic & Advanced cases

→ Basic Relative Xpath:

- 1) Xpath by Attributes
 - 2) Xpath by Text function
 - 3) Xpath by Partial Attribute
 - 4) Xpath by Partial Text function
- } Xpath by contains-function
↓
LinkText

i) Xpath By Attributes:

Syntax:

`//tagname[@AN='AV']`

Eg: `//input[@id='small-searchterms']`

ii) Xpath By text function.

Syntax: `//tagname[text()='textfield']`

Eg: `//input[text()='Register']`

∴

iii) Xpath By Partial Attribute.

Syntax: `//tagname[contains(@AN,'AV')]`

Eg: `//input[contains(@class,'search-buttons')]`

09/APR/24

-Advance X-path:

- ⑤ Xpath by forward traversing.
- ⑥ Xpath by backward traversing.
- ⑦ Xpath by following sibling.
- ⑧ Xpath by preceeding sibling.
- ⑨ Xpath by Start with.
- ⑩ Xpath by Multiple Attribute
- ⑪ Xpath by index.
- ⑫ Xpath by parent.
- ⑬ Xpath by child.
- ⑭ Xpath by Descendent.
- ⑮ Xpath by Ancestor.
- ⑯ Xpath by Self.
- ⑰ Xpath by Surrounding.

→

① Xpath by Attribute.

→ // tagName [@AN = 'VALUE']

② Xpath by LinkText

→ // tagName [text() = 'text-function']

③ Xpath by Partial Attribute

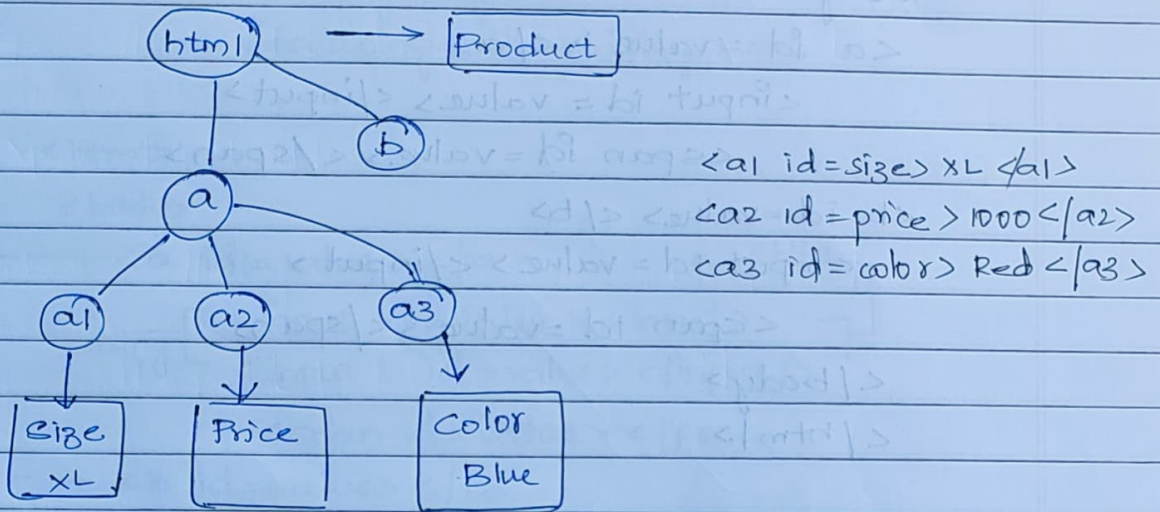
→ // tagName [contains (@AN, 'AV')]

④ Xpath by Partial Text Function

→ // tagName [contains (text(), 'text-function')]

⇒ Xpath by forward traversing:

• 'Moving' from upto down in a parent child relationship is called forward traversing.



<html id=product> t-shirt </html>

Scenario-1 → get the size of T-shirt.

```
String size = driver.findElement(By.xpath("//html[@id='product']/a/a1")).getText();
```

Scenario-2 → get the price of t-shirt.

```
String price = driver.findElement(By.xpath("//html[@id='product']/a/a2")).getText();
```

Scenario-3 → get the color of the t-shirt.

```
String color = driver.findElement(By.xpath("//html[@id='product']/a/a3")).getText();
```

Sopln(price); | Sopln(size) || Sopln(color)

10/APR/24

⇒ Xpath By Backward Traversing:

<html>

<body>

<input id = value > </input>

<b id = value >

<input id = value > </input>

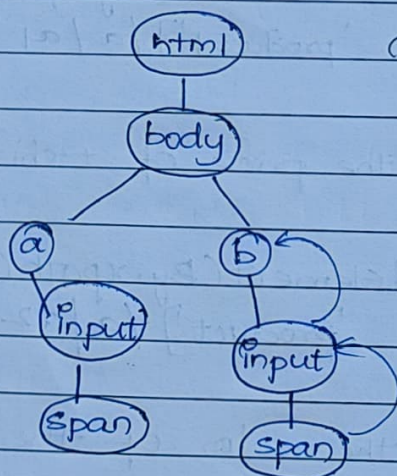
</body>

</html>

Def: Moving from down to up in a parent child relationship is called xpath by backward traversing.

• It is performed by using `/..`.

HTML Tree:



NOTE: We can only do this in parent child tags but not for sibling tags.

`//span[@id = 'value'] /.. /..`

① `//html/body/a/input/span /.. /..` :

→ Xpath by following Sibling:
Moving from elder sibling to younger sibling is known as xpath by following sibling.

Syntax: Elder to younger.
/following-sibling::tagname

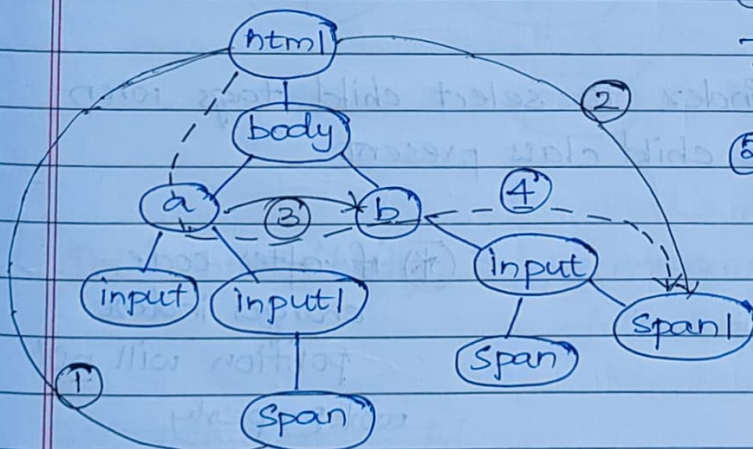
```

<html>
  <body>
    -----<a id = value> </a>
    sibling [
      <input id = value> </input>
      <input1 id = value> </input1>
      <span id = value> </span>
    ]
    -----<b id = value> </b>
    sibling [
      <input id = value> </input>
      <span id = value> </span>
      <span1 id = value> </span1>
    ]
  </body>
</html>

```

Annotations: "child" points to the first <a> and the first <input> under . "Sibling" is indicated by dashed lines connecting the <a> to its siblings and the to its siblings.

Html Tree:



① → html/body/a/input/
following-sibling::input1/span

② → html/body/b/input/following-sibling::span1

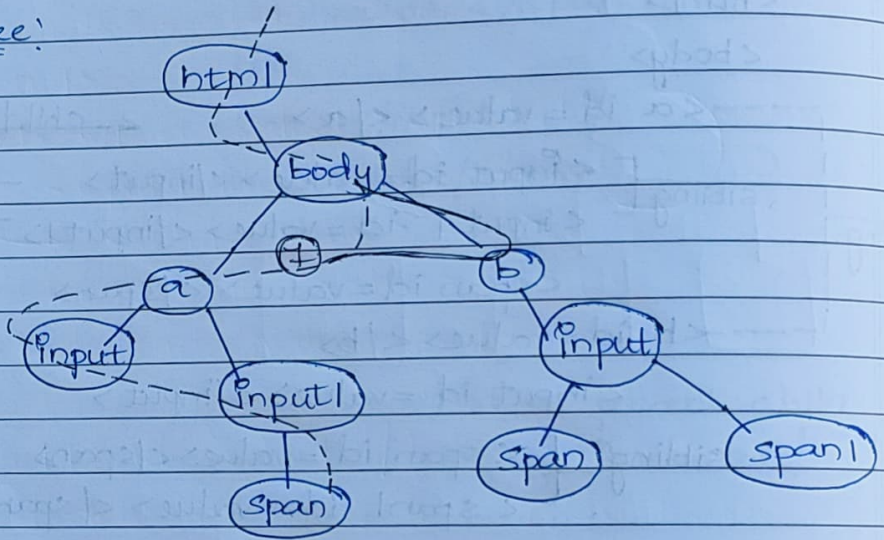
③ → html/body/a/following-sibling::b

④ → html/body/a/following-sibling::b/input/span

=> xpath by preceding sibling:
moving from younger sibling to elder sibling is known as xpath by preceding sibling.

Syntax: Younger to Elder.
/preceding-sibling::tagname

HTML-Tree:



① ⇒ `html/body/b/preceding-sibling::a/input/-following-sibling::input1/span`.

NOTE:

We can also use index to select child tags when there are multiple child class present
i.e.,

* if after code changes index position will not work properly

∴ But it is not a good/recommended practice

⇒ xpath by child:

Traversing between parent to child by using `"/child::"` is known as xpath by child.

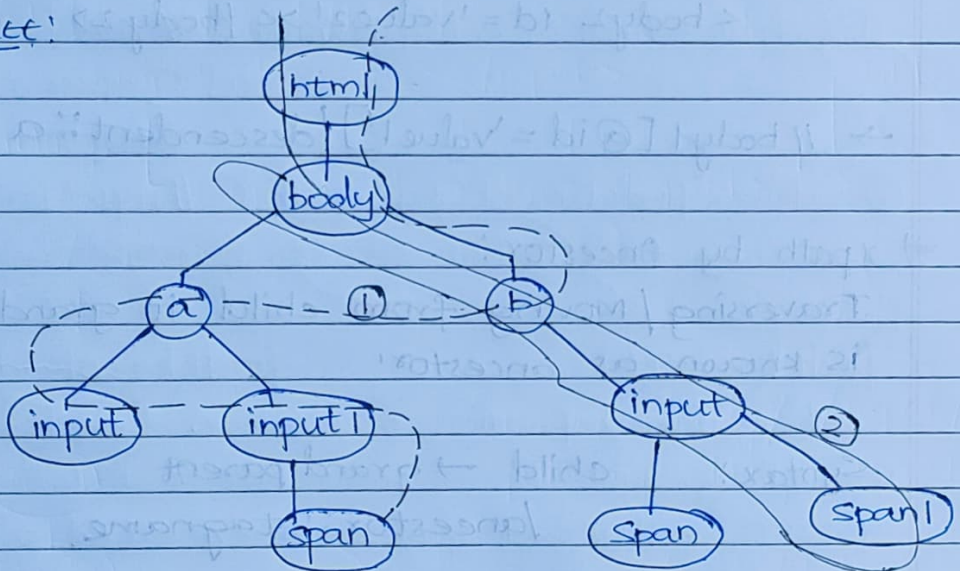
Syntax: `/child::tagname` [Parent → child]

⇒ xpath by parent:

Traversing between child to parent by using `"/parent::"` is known as xpath by parent.

Syntax: `child → parent:
/parent::tagname.`

HTML TREE:



① `html/body/b/preceding-sibling::a/input/following-sibling::input1/span`

~~② `html/body/b/`~~

② `/html/body/child::b/child::input/child::span1/parent::input/parent::b/parent::body`

→ xpath by Descendent:

Moving from parent to grandchild is known as descendent.

Syntax: Parent → Grandchild
/descendent::tagname.

Ex:

```
<html>
  <body1 id='value1'> </body1>
    <input1 id='value'> </input1>
    <a id='value'> </a>
    <input2 id='value'> </input2>
  <body2 id='value2'> </body2>
```

Diagram illustrating the relationship between elements:

- descendent:** An arrow points from `<body1 id='value1'>` down to `<a id='value'>`.
- ancestor:** An arrow points from `<a id='value'>` up to `<body1 id='value1'>`.

→ `// body1 [@id='value1']/descendent::a.`

→ xpath by Ancestor:

Traversing / Moving from child to grandparent is known as ancestor.

Syntax: child → grand parent
/ancestor::tagname.

→ `// a [@id='value']/ancestor::body1.`

→ xpath by startswith:

This xpath will work like contains method present in java

~~Syntax:~~ (ii) It starts with take the partial matching from the starting where as contains takes the matching from anywhere

Syntax: // tagname [starts-with (@AN, 'AV')]

↳ for attribute

// tagname [starts-with (text(), 'text-function')]

↳ for text function.

Eg: <input id = 'input1 demo' > </input >

// ~~input~~[@id]

// input [starts-with (@id, 'input1')] ↳ for attribute..

11/04/24

→ Xpath by Multiple Attributes:
finding the element by using more than one attribute by xpath.

Syntax:

AND ⇒ //Tagname[@AN='AV' and @AN='AV']
Both conditions should be true

OR ⇒ //Tagname[@AN='AV' or @AN='AV']
Any one of the condition must be true

Eg:

① driver.findElement(By.xpath("//input[@id='value' and @name='value1']"));

② driver.findElement(By.xpath("//input[@id='value' or @name='value1']"));

findElement()

◦ return type of this is WebElement.

◦ findElement() will give you first matching of the web element

◦ If any of the element is not matching with webpage. then findElement() will throw 'NoSuchElementException' exception.

findElements()

◦ return type of this is List<WebElement>

◦ this method will give you all the matching of the web elements

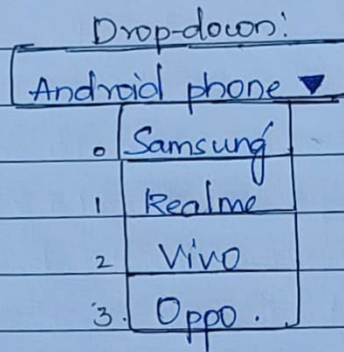
◦ If any of the element is not matching with web-page then findElements will give you [].

15/04/24

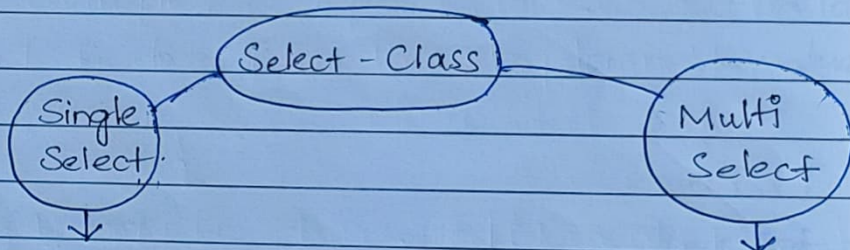
→ Select Class (Dropdown/ListBox)

Syntax: `Select s = new select(dropdown);`
 ↑
 web element.

Example:



```
<Select id="product-name"> Product </Select>
<option value="sam"> Samsung </options>
<option value="Rm"> Realme </options>
<option value="vv"> Vivo </options>
<option value="Op"> Oppo </option>
```



- 1) `Select By Index();`
- 2) `Select By value();`
- 3) `Select By VisibleText();`

- 1) `Select By Index();`
- 2) `Select By value();`
- 3) `Select By VisibleText();`
- 4) `Deselect By Index();`
- 5) `Deselect By value();`
- 6) `Deselect By VisibleText();`
- 7) `isMultiple();`
- 8) `getOptions();`
- 9) `getAllSelectedOptions();`
- 10) `getFirstSelectedOption();`
- 11) `DeselectAll();`

⇒ Points on select class:

- Select class is used to handle dropdown boxes.
- It accepts webdriver as an argument.
- It contains 11 methods.

{ Extra Space }

⇒ (i) Single Select ();

Single select dropdown can able to select one option at a time we cannot able to select more than one option.

→ We can't deselect the option which we have selected from dropdown

⇒ (ii) Multi Select ();

We can able to select multiple options at a time

→ We can able to deselect the option which is present in dropdown

{ LEAVE SPACE }

- Every dropdown webelement created using 'select' tag.
- The content of the dropdown will be created using 'option' tag.
- To handle dropdown we use the select class of Selenium.

- Select class has a parameterised constructor it accepts an argument of webelement type. ("Address of dropdown element")

Q. How to create object for Select class?
= `Select s = new Select (webelement);`

① `Select ByIndex ();`

- It is a method which comes from the select class.
- It is a non-static method.
- We can able to access the method with the help of select class obj reference.
- `SelectByIndex` is a method which accept 'int' type of an argument.

For Example:

⑤ Select Realme from the dropdown by using index method.

```
{  
webelement E = driver.findElement(By.Id("drop"));  
Select s = new Select (E);  
s.SelectByIndex(2);  
}
```

② `SelectByvalue ();`

- It is a method which comes from the select class.
- It is a non-static method.
- We can able to access the method with the help of select class obj reference.

• It accepts string type of an argument.

For Example:

⑤ Select Realme from dropdown by using value method.

```
{
    Select s = new Select(E);
    s.SelectByValue("Realme");
}
```

③ SelectBy VisibleText();

• ~~It can~~ It is a method which comes from the select class

• It is a non-static method.

• We can able to access the method with the help of select class obj reference.

• It accepts string type of an argument.

For Example:

⑤ Select samsung from dropdown by using Visible Text method.

```
{
    Select s = new Select(E);
    s.SelectByVisibleText("Samsung");
}
```

④ DeSelect By Index();

• We can able to deselect the method which is been selected before by providing index position.

• It accepts 'int' type of an argument.

⑤ → `DeSelect.By value()`;

- It accepts string type of argument
- It is used to deselect the selected value by providing its value as an argument.

⑥ → `DeSelect By VisibleText()`;

- It accepts string type of argument
- It is used to deselect the selected value by providing the link text of that particular value.

⑦ → `isMultiple()`;

- It is a method which is present in the `select` class.
- It accepts no arguments.
- The return type of `isMultiple()` is boolean
- It is used to check whether the selected dropdown is a multi-select dropdown or not.

⑧ → `getAllSelectedOptions()`;

- It accepts no arguments
- The return type of `getAllSelectedOptions` is `List<webElements>`
- It is used to get the elements which have been selected in the dropdown.

⑨ → `getFirstSelectedOption()`;

- It accepts no arguments.
- The return type of this method is "webElement"
- It is used to get the selected value which is present in the 1st position of the dropdown box.

⑩ `deselectAll();`

- It is a no argument method.
- The return type of `deselectAll` method is `void`.
- It is used to deselect all the options which have been selected in the dropdown box.

⑪ `getOptions();`

When the page loads, the user can select an option from the dropdown box. If the user selects an option, the value of the selected option is stored in the `selected` property of the `options` object.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

The `getOptions()` method is used to get the value of the selected option. It returns an array of objects, where each object represents an option in the dropdown box.

18/4/24

wait. condition :

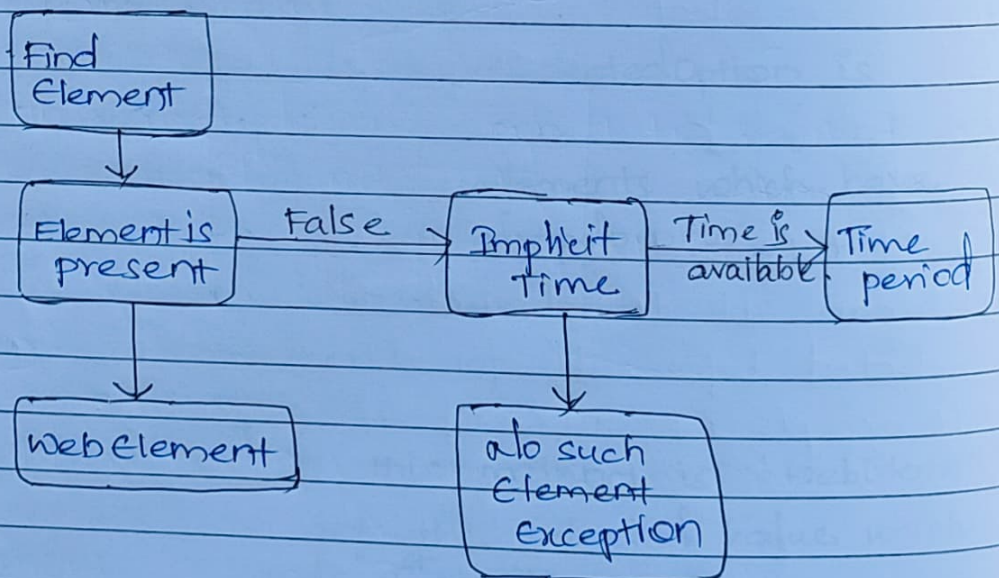
- i) Implicit wait
- ii) Explicit wait.

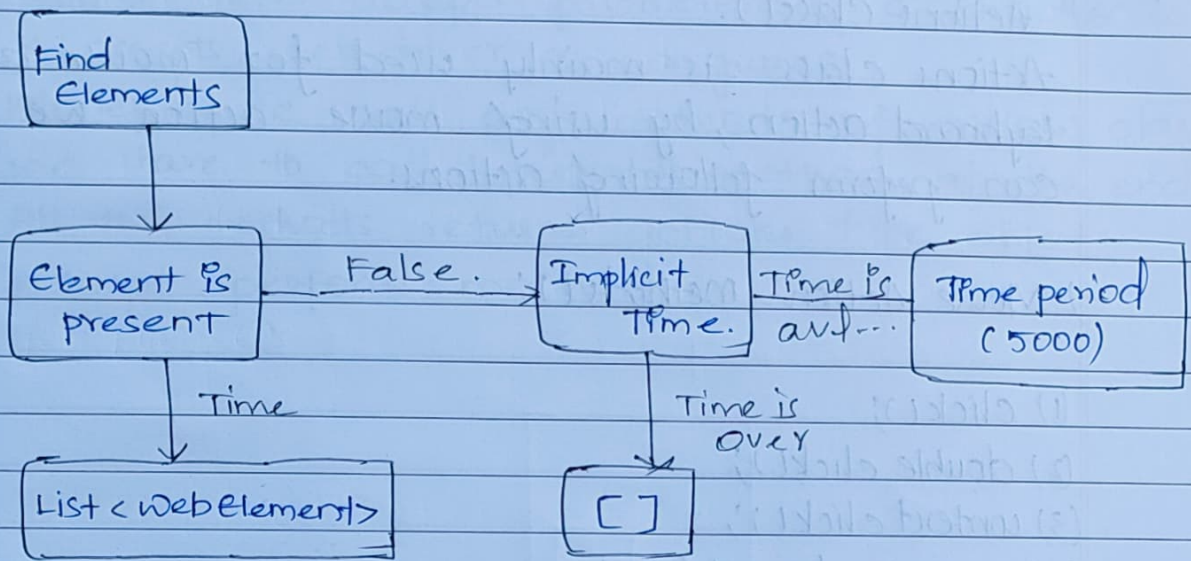
⇒ Synchronization/webdriver wait.

- The process of matching the selenium script with the application script is called synchronization.
- When the page loads on browser various web element on it which may load at different interval.
- So we make use of wait command to direct the test script to pass for a certain before throwing an exception.
- We go for wait command to reduce the usage of thread.sleep.

⇒ Types of wait command.

- i) Implicit wait
- ii) Explicit wait.





- Implicit wait will work for `findElement` and `findElements` method.
- When the control comes to `findElement` or `findElements` method it will check whether the element is present or not.
- If the element is present then the `findElement` method will return the first matching of web-element whereas `findElements` will return the all matching of the web element.
- If the element is not present then it will check for implicit time provided.
- If the time is available it will wait for the particular period of time.
- If the time is over then `findElement` method throw you `NoSuchElementException`.
where as `findElements` method return the `[]` empty list.

19/04/24.

→ Actions class():

Actions class is mainly used for mouse and keyboard action, by using mouse action we can perform following actions.

→ Mouse Action methods():

- (1) click();
- (2) double-click();
- (3) context-click();
- (4) move to element (WB)
- (5) click(WB);
- (6) double-click(WB)
- (7) context-click(WB)
- (8) drag and drop (WB, WB)
- (9) click-and-hold()
- (10) release()
- (11) click-and-hold(WB)
- (12) release(WB)
- (13) sendkeys
- (14) build()
- (15) perform.

→ All these methods are non-static methods it presents inside the actions we have to create object for that particular class.

Syntax:-

```
WebDriver driver = new ChromeDriver();
```

```
Actions a = new Actions(driver);
```

- Actions class accepts parameterized constructor where it accepts web driver as argument.
- Whenever we call any methods of actions class we have to call perform method at the end
- All the methods returns actions type object except perform method.

22/04/24

⇒ Pop-up:

pop ups are appears on the window when end user perform some actions in web pages.

Purpose of pop-up:

- (i) To grab attention of end user.
- (ii) To give information for the end user.
- (iii) To collect information from end user.

Pop-ups are categorized into following types:

- (i) Alert/Javascript executor.
- (ii) Hidden vision popup.
- (iii) File upload pop-up.
- (iv) File download pop-up
- (v) Notification popup
- (vi) Authentication pop-up
- (vii) Child window pop-up.

(1) Alert/Javascript executor pop-up:

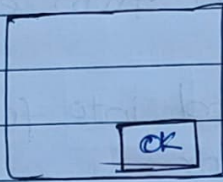
- These pop-up are developed by using 'javascript language'.
- we cannot avoid this popup.
- Alert is Interface.

Types of Alert/Javascript executor pop-up:

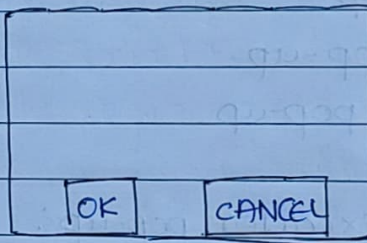
- (i) Alert/Simple Alert popup.
- (ii) Confirmation popup
- (iii) prompt popup.

Characteristics of Javascript Executor popup:

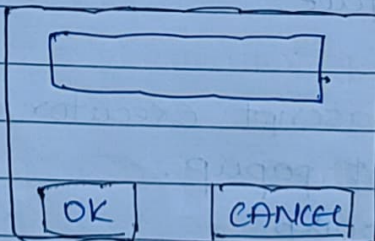
- (1) we cannot inspect this popup.
- (2) we cannot move this popup.
- (3) We cannot perform any action on the web page unless we handle this popup.
- (4) The popup will have only ok button is called Alert/Simple alert popup.



- (5) The popup which contain ok and cancel is called as confirmation popup.



- (6) The popup which contain text field is called as prompt popup.



To handle Javascript pop up:

Syntax: Action a = driver.switchTo().Alert();

(1). Accept()

(2) dismiss()

(3) SendKeys()

(4) getText()

NOTE:

◦ without handling javascript popup. if we are performing any action then we will get "UnhandledAlertException".

◦ If a popup contains only ok button then there is no difference between accept or dismiss method. Both will click 'ok'.

◦ If we try to switch drive control to alert popup before it appears we will get "NoAlertPresentException".

(1) accept();

This method is used to click on OK button in the popup.

(2) dismiss();

This method is used to click on "CANCEL" button in the popup.

(3) sendKeys();

This method is used to send the data to the textfield.

(4) `getText()`;

This method is used to get the text of the pop.up.

⇒ Javascript Executor:

- ° Javascript executor is an interface which provides the mechanism to write the java-script code in the selenium webdriver.

- ° In case find elements and locators is not working, we can use javascript executor to perform an action on a web element.

- ° It provides 2 abstract methods:

- (1) `executeScript()`;

- (2) `executeAsyncScript()`;

- ° These 2 methods are implemented in remote web driver class.

- ° To use this methods we should cast the webdriver object to javascript executor type.

Syntax: `JavascriptExecutor js = (JavascriptExecutor) driver;`

`js.executeScript();`

⇒ Operations performed by J.S Executor:

- (1) window scrolling.

- (2) performing action of disabled text

- (3) performing action on hidden element

In javascript by default 2 global object will be available:

- (i) window object represented by reference variable of window().
- (ii) document object represented by reference variable of document.
- Using this 2 global object we can invoke functions in javascript.

⇒ To scroll the window:

There are 3 ways:

- (i) ScrollBy (x, y);
- (ii) ScrollTo (x, y);
- (iii) scrollIntoView (boolean);

("argument[0].scrollIntoView (t/f)", element);

webelement

* To scroll the window to extreme bottom of webpage.

(1) scrollTo (0, document.body.scrollHeight);

* To scroll the window to extreme top of webpage.

(1) scrollTo (0, document.body.scrollTop);

** To remove the attribute.

js.Executescript ("arguments[0].removeAttribute ("disabled"), "element");

↓
webelement.

→ Performing action on hidden element.

```
package javascriptExecutor
```

```
import java.time.Duration;
```

```
public class HiddenText {
```

```
    p.s.v.m ( )  
    {
```

```
        WebDriver driver = new ChromeDriver();
```

```
        driver.manage().timeouts().implicitlyWait
```

```
            (Duration.ofSeconds(20));
```

```
        driver.get("https://www.facebook.com/signup");
```

```
        JavascriptExecutor js = (JavascriptExecutor) driver;
```

```
        js.executeScript("document.getElementById
```

```
            ('custom_gender')[0].
```

```
            value = 'male'");
```

```
    }
```

```
}
```

Scenario:

Performing action on hidden element present in facebook signup page.

Take Screenshot();

- It is an Interface which includes the mechanism to take screenshot.
- It has 2 different Implementation
 - 1) RemoteWebElement (Screenshot of particular webelement)
 - 2) RemoteWebDriver (Screenshot of entire webpage)
- ° It provides one abstract method which is ~~getScreenshot~~ getScreenshotAs()
 - (i) getScreenshotAs();
- ° Return type is based on the argument passed inside the getScreenshotAs()
 - (i) getScreenshotAs (OutputType.File);
 - (ii) getScreenshotAs (OutputType.Bytes);
 - (iii) getScreenshotAs (OutputType.Base64);

NOTE:

It is the good practice to take screenshot inside a folder called screenshot or errorshot.

- ° whenever there is a defect, it is a best practice to add a screenshot in default report.
- ° whenever any automation script failed we take screenshot.

26/04/24

classmate

Date
Page

⇒ How to Take Screenshot :

- `getScreenshot()` will take the screenshot and store it in a temporary directory and the screenshot will be deleted after the program ends.
- So we should move the file to a different location before the program ends.
- To move the file to the different location we use `'Files.copy()'`.

⇒ Ex-1: To take screenshot of the webpage:

- Step 01: `TakeScreenshot ts = (TakeScreenshot) driver;`
- Step 02: `File source = ts.getScreenshotAs (OutputType.FILE);`
- Step 03: `File destination = new File ("filename.png");`
- Step 04: `Files.copy (source, destination);`

⇒ Ex-2:

To take screenshot of the web element:

• Step 01:

~~TakeScreenshot ts = (TakeScreenshot)~~

`WebElement element = driver.findElement (By.id (" "));`

• Step 02:

`File source = element.getScreenshotAs (OutputType.FILE);`

• Step 03:

File destination = new File ("filename.png");

• Step 04:

Files.copy (Source, destination);

⇒ iFrame:

A webpage inside the webpage is called embedded webpage.

⇒ purpose of webpage:

1) Advertisement

2) Captcha

3) To merge code inside the common page. If two or more developers working on a same page.

Syntax: driver.switchTo.frame();

Frame Method:

It is an overloaded method we can switch our window driver control from parent window to frame in 3 ways.

i) driver.switchTo.frame (int);

ii) driver.switchTo.frame (string);

iii) driver.switchTo.frame (webelement);

NOTE:

To switch back the driver control we have two methods.

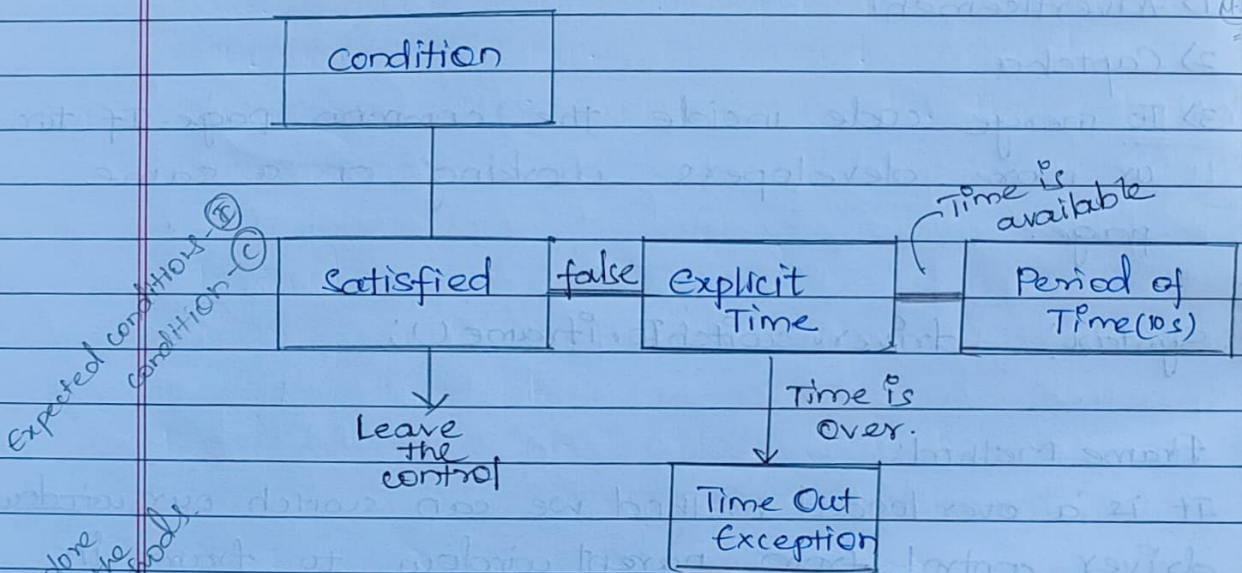
(i) `driver.switchTo.parentFrame();`

• To switch back the driver control to Immediate parent frame.

(ii) `driver.switchTo.defaultContent();`

• To switch back control to the main page.

⇒ Explicit wait():



Syntax:

```

WebDriverWait wait = new WebDriverWait(driver,
    Duration.ofSeconds(20));
wait.until(ExpectedConditions.titleContains("demo web shop"));
  
```

↓
method present in expected condition.

- 1) In order to handle the synchronization of any method we can use explicit time.
- 2) `WebDriverWait` itself is called an Explicit wait. Because we have to specify the waiting condition explicitly.
- 3) When the control comes to "`wait.until`" statement. It will check the specific condition.
- 4) If the condition is true it will go to the next statement.
- 5) If the condition is false it will check for explicit time provider.
- 6) If the time is over it will throw "Time Out Exception".

NOTE:

- `ExpectedCondition` is an Interface. where as `ExpectedConditions` is a class.

Hidden Division pop-up

→ Characteristics:

- we cannot move this pop-up
- we can able to inspect this pop-up

→ To handle:

- we can make use of `findElement()`.

→ File Upload Popup:

- we can move this pop-up.
- we cannot inspect this pop-up
- This pop-up contains open and cancel button
- we can either avoid or handle this file upload pop-up.

⇒ To Avoid:

- Identify input tag which contains an attribute `"type=file"`.
- And make use of `sendKeys("absolute path file")`

→ To Handle:

- By using Robot class

⇒ File Download pop-up:

By default it will be avoided by the selenium.

⇒ Notification Popup:

- We cannot move this popup.
- We cannot inspect this popup.
- This popup contains allow and block button.
- It will be displayed below the address bar in the beginning.
- In order to avoid this popup we can change the browser setting. So that Notification popup will not be displayed.
- To change the setting of the browser, we use add arguments method of ChromeOptions class.

Syntax: ~~ChromeDriver~~

ChromeOptions setting = new ChromeOptions();

setting.addArguments("--disable-notifications");

WebDriver driver = new ChromeDriver(setting);

OR ELSE WE CAN PASS:

setting.addArguments("--incognito");

⇒ To Handle:

By using "Robot" class we can handle:

⇒ Authentication Pop-up:

(Extra-Space)

Characteristics:

- It we cannot move the popup.
- we cannot inspect the popup.
- This popup will have username and password text box along with sign in and cancel button.

To Avoid:

To avoid this popup by sending 'username and password along with the URL. inside the get method

`https://username:password@www.demowebshop.com`

To handle:

By using Robot Class:

→ child window popup:

(i) We can move this popup.

(ii) We can inspect this popup.

(iii) Every child window contains maximize, minimize and close button along with the address path.

To handle:

We can make use of `getWindowHandle()` or `getWindowHandles()` or `driver.switchTo.window()`;

Pop-ups:

	<u>MOVE</u>	<u>INSPECT</u>	<u>AVOID</u>	<u>HANDLE.</u>
ALERT	X	X	X	<code>switchTo().Alert</code>
HIDDEN	X	✓	X	<code>findElement()</code>
FILE UPLOAD	✓	X	AttributeType = "file"	Robot class
FILE DOWNLOAD	X	X	X	✓
NOTIFICATION	X	X	chromeOptions	Robot class
AUTHENTICATION	X	X	Password Username	Robot class
CHILD WINDOW.	✓	✓	X	<code>switchTo().window()</code> <code>getWindowHandle()</code> <code>getWindowHandles()</code>

TestNG

- TestNG stands for Test Next Generation.
- It is a unit testing framework tool. Basically used by a developer to perform unit testing.
- we automation engineer use testing for:
 - (i) Report Generation
 - (ii) Batch Execution
 - (iii) Parallel Execution
 - (iv) Group Execution.

Rules to follow while using TestNG class:

- 1> Use @Test instead of main method.
- 2> Use Reporter.log instead of println statement.

@Test:

- 1> @Test Annotation is Java Template which is used to give information to compiler and runtime environment.
- 2> Annotation can be used on top of the variables, methods and class.
- 3> @Test is a core annotation of TestNG without which cannot run any TestNG program.
- 4> Any class which consists atleast one @Test annotation method is called "TestNG class".
- 5> Any method which is annotated with @Test annotation is considered as a TC and code written within this method is considered as Test Scripts.

TESTNG. Helper

(i) Priority:

- It determines the sequence of the execution of the T.C.
- The lowest priority of the T.C will run first and the highest priority of the T.C will be executed at the last.
- If the priority is not specified then the default priority will be zero.

Example:

@Test (priority = 2)

NOTE:

- We go for priority to execute the test method / test data/cases in required order / user-defined order.
- If priority is duplicate then it will be executed in alphabetical order.

* Invocation Count:

- It is used when we want to run the same T.C. multiple times.
- By default invocation count is '1'.

Ex:

- If you want to run the single @Test case for 10 times.

@Test (Invocation Count = 10)

* ThreadpoolSize:

- It is used for parallel execution when we want to execute same T.C. multiple times in parallel then we go for threadpoolSize.
- It works only when InvocationCount attribute is used.
- By default threadpoolSize is '0'.

Ex!

```
@Test (Invocation count = 10, threadpoolSize = 5)
```

* Enabled:

- It contains the boolean values.
- By default its value is true.
- If you want to skip some test cases then we need to explicitly specified 'false' value.

Ex! @Test(enabled);

* dependsOn methods:

- It is used to create the dependency between the two test cases.
- It will wait till the dependent module is executed successfully and then only execute the particular method/module.
- The scope of dependency method is within the particular TestNG class.

Ex! @Test (dependsOn method = "Create Account")
 public void DeleteAccount() {

}

@Test

```
public void CreateAccount CI {  
    }  
}
```

TestNg Annotation:

- 1) @Before Suite → Connection to the server
- 2) @After Suite → Dis-connect to the server
- 3) @Before Test → Connect to the database
- 4) @After Test → Disconnect to the database
- 5) @Before class → Open the Browser
- 6) @After class → Close the Browser
- 7) @Before method → Login
- 8) @After method → Logout
- 9) @Test → Test cases

* Chronological Order:

@Before Suite

@Before class

@Before method

@Test

@After method

@After class

@After test

@After Suite

⇒ Batch Execution:

- Executing multiple test cases (or) test scripts in one shot is called Batch execution
- Execution all the class together via testNG.xml file.

⇒ How to create .xml file:

- 1) Select all the testNG classes
- 2) write click go to testNG options and select convert to testNG
- 3) Click on finish.

◦ xml.

```
< suite
```

```
  < test
```

```
    < classes
```

```
      package Name. class Name
```

```
      package Name. class Name
```

```
    </ classes>
```

```
  </ test>
```

```
</ suite>
```

⇒ Parallel Execution:

- Execution multiple test cases simultaneously (or) parallelly is called parallel execution
- In xml file at test level provide parallel = true

```
parallel = "true";
```

⇒ Group Execution:

- The name of the group is to be mention
- We can group multiple methods under a test suite using .xml file.
- The group should be done inside the test tag before the class tag.

```
<groups>
  <run>
    <include name="smoke"> </include>
  </run>
</groups>
```

⇒ Data providers:

- To execute the same test cases multiple times with the different set of test data we make use of @DataProviders Annotation
- Sending data from data provider to test case is called dependency injection.

```
Ex! @Test (dataProvider = "credential")
public void testcase (String s1, String s2) {
  Reporter.log (s1, true);
  Reporter.log (s2, true);
}

@DataProvider (name = "credential")
public Object [][] testData () {
  Object [][] d = new Object [2] [2];
  d [0] [0] = "SQL";
  d [0] [1] = "Java";
}
```



```
d[0][0] = "MT";  
d[0][1] = "AT";  
return d;  
}
```

⇒ Assertions:

- Assertions are basically libraries (or) functions which are used to verify certain critical check points of the test cases.
- Asserts in selenium validate the automated test cases that help testers understand if test have passed (or) fail.
- We go for assertion to reduce the lines of code.

⇒ Types of Assert:

- Assert is classified into 2 types:

(i) Hard Assert

(ii) Soft Assert.

⇒ Hard Assert:

- If ~~there~~ there is any failure it will terminate the execution.
- Use assert equals method which present in the assert class.

Syntax:

Assert.assertEquals (Expected value, Actual value);

= Soft Assert:

- If there is any failure it will not terminate the execution.
- Use `assertEquals()` which present in the `soft assert` class.

Syntax:

```
SoftAssert s = new SoftAssert();
s.assertEquals(Expected value, Actual value);
s.assertAll();
```

NOTE:

- At the end of the script call `assertAll()` to show the assertion failure.

- Assert (Hard Assert)

1) All the methods are static methods →

2) In case of failure test cases will be terminated suddenly

3) No need to call any method to show assertion error

- Soft Assert

1) All the methods are non-static methods

2) In case of failure test cases will not be terminated suddenly

3) We need to call `assertAll()` to show the assertion error

⇒ Cross Browser Testing:

Executing the TC or Test scripts in different browser is called as Cross Browser Testing.

- It is also known as compatibility testing.
- In TestNG class we use @parameter annotation over the @Before class.
- In suite file we use parameter tag to pass the data to the TestNG class.

⇒ ITest Listener:

- It is a feature available in TestNG which is used to capture the runtime events during the execution and perform appropriate action based on given time.
- "@Listeners" is a TestNG annotation used to monitor the test execution in runtime and generate runtime event.

⇒ ITest Listener: (I)

It is an Interface in TestNG used to capture test runtime event.

- We should write Listener implementation class to use "@Listener" annotation.
- ~~Listener~~ in
- Listener implementation class should implement ITestListener and override required abstract method.

@Listener (Listener Implementation.class)

class Base

{

@Before class

public void openBrowserUI{

=====
}

@After class

public void closeBrowserUI{

{
=====
}

class program

{

@Test

public void TC1{

=====
}

ITest Listener <<I>>

class Listener Implementation implement ITestListener

{

@Override

public void onTestStart (ITestResult result)

{

Reporter.log("Test Execution started", true);

}

@Override

public void onTestFailure (ITestResult result)

{

Reporter.log("Test Execution failed", true);

}

@Override

public void onTestSuccess (ITestResult result)

{

Reporter.log("Test Execution Successful", true);

}

ABSTRACT

- 1) OnTestStart
- 2) OnTestFailure
- 3) OnTestSuccess

09/MAY/24

```
@Test(dataProvider = "credential")
public void test(String trainer, String subject)
{
```

```
    Reporter.log(trainer, true);
```

```
    Reporter.log(subject, true);
}
```

```
@dataProvider("credential")
```

```
public Object[][] testData() {
```

```
    FileInputStream fis = new FileInputStream
        ("filepath.xls");
```

```
    Workbook w = WorkbookFactory.create(fis);
```

```
    Sheet s = w.getSheet("Sheet1");
```

```
    int rowsize = s.getPhysicalNumberOfRows();
```

```
    int cellsize = s.getRow(0).getPhysicalNumberOfCells();
```



1	Prem	Java
2	Yogesh	SQL
3	Thulasi	Firelink

OBJECT

0	0	1
0	Prem	null Java
1	Yogesh	null SQL
2	Thulasi	null Firelink

i	j	i < rowsize	j < cellsize
0	0	0 < 3 → T	0 < 2 → T
0	1		1 < 2 → T
0	2		2 < 2 → F
1	0	1 < 3 → T	0 < 2 → T
	1		1 < 2 → T
	2		2 < 2 → F
2	0	2 < 3 → T	0 < 2 → T
	1		1 < 2 → T
	2		2 < 2 → F
3	0	3 < 3 → F	

```
Object[][] d = new Object[rowsize][cellsize];
```

```
for (int i = 0; i < rowsize; i++) {
```

```
    for (int j = 0; j < cellsize; j++) {
```

```
        d[i][j] = s.getRow(i).getCell(j).toString();
```

```
    }
}
```

```
return d;
```